LUXE & THREAD · DOCUMENTATION

INTERNAL – for developers

Internal 09 · Method of Procedure (MOP) – Training Guide for New Team Members

Audience	INTERNAL – a new developer / operator joining the team
Goal	After following this guide you can run, change, test, release and support the shop on your own
How to use	Do the procedures **in order**, typing each command. Every command is explained. Tick the boxes.
Time	Day 1: §1–5 · Day 2: §6–9 · Day 3: §10–13

> **Golden rules:** (1) Never experiment on the live server. (2) Never paste a secret into chat/e-mail/screenshots. (3) Back up before changing data. (4) If unsure, ask – a 2-minute question beats a 2-day outage.

---

1 · Day-1 setup (your computer)

1.1 Install tools | Tool | Check | Why | |---|---|---| | Node.js 22 LTS | `node -v` → `v22.x` | runs the app | | Git | `git --version` | version control | | Docker Desktop | `docker --version` | runs PostgreSQL locally without installing it | | VS Code | – | editor (+ extensions: ESLint, Tailwind CSS IntelliSense) | | psql (PostgreSQL client) | `psql --version` | look inside the database |

1.2 Get the code

```
git clone <repo-url> fashion-store      # copies the project from GitHub onto your computer
cd fashion-store                        # go inside the folder
npm install                             # downloads all libraries listed in package.json
```

⚠️ Never copy `node_modules` from another computer; always run `npm install`.

1.3 Start the database

```
docker compose up -d db                 # starts PostgreSQL 16 in the background (see docker-compose.yml)
docker compose ps                       # status should be "healthy"
```

1.4 Settings

```
cp .env.example .env.local              # make your private settings file (never committed to Git)
```

Open `.env.local` and set `SESSION_SECRET` and `JOBS_SECRET` to long random text: run `openssl rand -hex 32` twice and paste. Leave Razorpay/Shiprocket empty for now (the site still runs).

1.5 Create the tables and demo data

```
npm run db:migrate                    # runs db/migrations/*.sql once each → creates all tables
npm run db:seed                       # loads demo products, sizes, stock and coupon WELCOME10
npm run admin:create -- me@test.in "A Long Passphrase 123" "Me" owner     # your admin login (min
12 chars)
```

1.6 Run it

```
npm run dev                           # development server → http://localhost:3000
```

✅ Checklist: shop opens · Women/Men/Kids work · `/admin/login` signs you in · `/api/health` shows `"database":true`.

2 · Understand the project in 30 minutes

Read in this order: External HLD (the idea) → Internal 02 LLD §1–4 (the design) → Internal 03 Database §1–4 (the data). Then open these files and read the top comment of each: `lib/pricing.ts`, `lib/orders/service.ts`, `lib/orders/status.ts`, `app/api/checkout/create/route.ts`.

3 · Folder map

Folder	Contains	You change it when…
`src/app`	pages and API	adding/changing a screen or endpoint
`src/components`	reusable UI	changing look of shared parts
`src/lib`	rules, services, SQL	changing business behaviour
`src/lib/__tests__`	automated tests	**every** time you change a rule
`db/migrations`	database structure	changing tables (new file only!)
`docs`	documentation	whenever behaviour changes
`tests`	e2e, integration, load	adding scenarios

4 · The four quality commands (run before every commit)

```
npm run typecheck     # TypeScript: finds wrong types/typos across the whole project
npm run lint          # ESLint: finds likely bugs and style problems
npm test              # unit tests of the business rules (seconds)
npm run build         # production build: catches Next.js-specific mistakes
```

All four must pass. If `npm test` fails read the message: it names the rule and the expected/actual values.

5 · Make your first change (practice)

1. Create a branch: `git checkout -b feat/my-first-change`.
2. In Admin (`/admin/products`) open a product and change its name, then reload the shop – you changed data.
3. Change code: in `lib/pricing.ts` find `SHIPPING_FEE = 0`, set `99`, run `npm test` → a test fails (it expects free shipping). That is the safety net working. Set it back to `0`.

4. Commit: `git add -A && git commit -m "docs: practice change"` (the pre-commit hook regenerates `docs/api`).

6 · Everyday procedures

6.1 Add a product (owner does this in Admin – you should know it)

Admin → Products → **+ Add product** → fill SKU (e.g. `LT-TST-900`), prices, sizes `S:5;M:5` → Create → upload photos. Verify in the shop.

6.2 Import new arrivals by CSV

1. Admin → Products → **Import CSV / photos** → *download template*.
2. Fill one row per product; sizes like `S:10;M:12`.
3. Upload photos named `SKU_1.jpg`, `SKU_2.jpg`.
4. Upload CSV → **Check file only** → fix reported line numbers → **Import now**. Rule to remember: *product CSV never changes stock of existing sizes*; use **Inventory → stock CSV** to refill.

6.3 Receive stock (refill)

Product page → *Stock by size* → type quantity under **Receive new stock** → Add. Or CSV `variant_sku,add_qty,note`.

6.4 Run a festival sale

Products → *Festival sale* → department/category, %, start/end dates → **Apply sale**. Undo with **Remove sale**. Check a product page shows the strike-through price.

6.5 Handle an order

Order arrives → Admin → Orders → open it → if status *Paid/COD* and no shipment → **Create shipment** → pack → courier picks up → tracking updates automatically (or **Sync tracking**).

6.6 Cancel / refund / return

See Internal 13 and Admin guide. Remember: cancelling restocks and refunds automatically; refunds need the **owner** role.

7 · Working with the database (read-only first)

```
psql "$DATABASE_URL"                                    # open an interactive session  (\q to
quit)
\dt                                                     # list tables
select sku, name from products limit 5;                 # look at data
select id, status, total from orders order by created_at desc limit 5;
```

⚠️ On production, only run `SELECT` unless a senior approves. Always `BEGIN;` … check … `COMMIT;` (or `ROLLBACK;`) for updates.

8 · Testing procedures

Level	Command	When
Unit	`npm test`	every change
Integration (real DB)	`DATABASE_URL=postgres://luxe:luxe_dev_password@localhost:5432/luxe_thread_test npm run test:integration` (create the `luxe_thread_test` database first: `docker compose exec db createdb -U luxe luxe_thread_test`)	DB/stock changes
End-to-end	`npm run build && npm start` in one terminal, `npx playwright install` once, then `npm run test:e2e`	before release
Load	`k6 run -e BASE=https://staging… tests/load/shop.k6.js`	before big sales

Details: Internal 11.

9 · Testing payments & shipping (staging)

Follow Internal 04: Razorpay TEST keys, webhook via tunnel (`npx ngrok http 3000`), Shiprocket with `SHIPROCKET_AUTO_CREATE=false`. Place a test order with card `4111 1111 1111 1111`; confirm Admin shows **Paid** and Messages shows **sent**.

10 · Release procedure

1. All four quality commands pass on `main`; docs updated.
2. Back up production DB (Internal 10 §3).
3. Tag: `git tag v2.0.1 && git push --tags`.
4. On the server:

```
cd ~/fashion-store
git fetch --tags && git checkout v2.0.1      # switch to the released version
npm ci                                        # install exactly the locked libraries
npm run db:migrate                            # apply new database changes (safe to re-run)
npm run build                                 # build for production
pm2 reload luxe-thread                        # restart without downtime
```

5. Run the Health Check §A.
6. **Rollback:** `git checkout <previous-tag> && npm ci && npm run build && pm2 reload luxe-thread` (database migrations are additive; if a migration must be undone, restore the backup taken in step 2 – ask a senior).

11 · Operating procedures

Procedure	Steps
Restart	`pm2 restart luxe-thread`, then `curl /api/health`
View logs	`pm2 logs luxe-thread` (Ctrl+C to stop)
Rotate a secret	change value in `.env.production.local`, `pm2 restart luxe-thread`, re-validate (Config guide §5)
Create staff login	`npm run admin:create -- staff@shop.in "Strong Passphrase 123" "Name" staff`
Disable a staff member	`psql "$DATABASE_URL" -c "update admin_users set is_active=false where lower(email)='staff@shop.in'"`

Procedure	Steps
Retry failed messages	Admin → Messages → Retry
Release unpaid orders now	`curl -X POST -H "x-jobs-secret: $JOBS_SECRET" https://host/api/jobs/expire-orders`
Update OS	`sudo apt update && sudo apt upgrade -y` then reboot in quiet hours; check `pm2 status`

12 · Incident procedure

1. **Stay calm, write the time.** 2. `curl /api/health`. 3. `pm2 status` + logs. 4. If a release caused it → rollback (§10.6). 5. Money safety: Razorpay records every payment – after recovery reconcile (Payments guide §6). 6. Tell the owner what happened and what is fixed. 7. Write a short report: cause, fix, prevention.

13 · Competency checklist (sign-off)

#	Can you…	✓
1	start the app locally with a database	☐
2	explain why prices are decided on the server	☐
3	add a product and receive stock	☐
4	import a CSV and fix a rejected line	☐
5	place a test order and trace it in `order_events`	☐
6	cancel an order and show the stock returned (`stock_movements`)	☐
7	run the four quality commands and read a failure	☐
8	take and **restore** a database backup	☐
9	deploy a release and roll it back	☐
10	diagnose "customer paid but order unpaid"	☐